cta-writer — Full Guide

Turn any short-form script into 3–5 ready-to-post CTAs, pick one, and get a paste-ready ManyChat setup that routes viewers to a specific long-form YouTube video.

What this eliminates:

- Scrolling a client's YouTube channel trying to find the right video for each Reel
- Writing a fresh CTA for every post
- Typing the same ManyChat automation fields from scratch every time
- Accidentally reusing a keyword ManyChat already owns

What the skill actually does

1. You paste a short-form script.
2. Claude reads your client's real YouTube transcripts (not just titles) and picks the long-form video that actually covers the same topic.
3. Claude returns 3–5 CTA options with a keyword that hasn't been used for this client yet.
4. You pick one.
5. Claude prints a paste-ready ManyChat block: public comment reply + the 3-stage DM sequence (opener → follow-gate → link), each with a random variation so viewers don't see the same automation twice.
6. You paste the fields into ManyChat → Go Live.

That's the whole loop.

What you need before you start

On your computer:

- Claude Code installed (free)
- Python 3.9 or newer
- `yt-dlp` — install with `brew install yt-dlp` on macOS, or `pip3 install --user yt-dlp`
- `youtube-transcript-api` — install with `pip3 install --user youtube-transcript-api`

On the platforms:

- A **ManyChat Pro** plan with your Instagram account connected
- For each client: their **YouTube channel URL** and their **Instagram account** connected to ManyChat

---

Install the skill

One command. Claude Code auto-loads anything under `~/.claude/skills/`.

```
git clone https://github.com/jetsuthpring/cta-writer.git ~/.claude/skills/cta-writer
```

Verify it's loaded by opening Claude Code and checking that `/cta-writer` appears when you start typing a slash command.

Register each client (one-time per client)

Two commands: register them, then scrape their channel.

1. Add the client

```
python3 ~/.claude/skills/cta-writer/scripts/add_client.py "Client Name" https://www.youtube.cc
```

Example:

```
python3 ~/.claude/skills/cta-writer/scripts/add_client.py "Dyck" https://www.youtube.com/@koeh
```

Output:

```
Added client: Dyck (dyck) -> https://www.youtube.com/@koehndyck
Next: run update_videos.py dyck
```

The `(dyck)` part is the slug — you'll use it in every other command.

2. Scrape their videos + transcripts

```
python3 ~/.claude/skills/cta-writer/scripts/update_videos.py dyck
```

First run on a large channel takes a few minutes. Each video gets a metadata fetch (via `yt-dlp`) plus a transcript fetch (via `youtube-transcript-api`). Future runs are incremental — only new uploads get fetched, so they finish in seconds.

You'll see live progress like:

```
[dyck] Listing videos from https://www.youtube.com/@koehndyck ...
[dyck] Channel has 27 videos; 0 cached.
[dyck] Fetching metadata for 27 new videos...
  [1/27] meta: FREE High Ticket Sales Course For 2026 (4+ hours)
  ...
[dyck] Fetching transcripts for 27 videos...
  [1/27] transcript: FREE High Ticket Sales Course For 2026 (4+ hours)
  ...
[dyck] Saved 27 videos (27 with transcripts) -> data/videos/dyck.json
```

Done. Your client is ready.

Re-run `update_videos.py` whenever your client uploads a new long-form.

Either do it manually when you remember, or right before drafting CTAs (the skill will remind you). It's fast unless there are truly new videos.

The daily workflow: script → CTA → ManyChat block

Step 1 — Invoke the skill with a script

Open Claude Code. Type `/cta-writer` and paste your short-form script. Tell Claude which client it's for.

Example:

```
/cta-writer

Client: dyck

Hook: "What do you say when they hit you with 'I need to think about it'?"

Body: Don't say "what do you need to think about?"... (full script)

CTA: "Follow for more objection scripts."
```

Step 2 — Read the output

Claude will:

1. Auto-refresh the client's video cache (fast).
2. Shortlist candidate videos by title + description.
3. Read the transcripts of just those candidates to confirm the match.
4. Return something like:

```
Matched long-form:  FREE High Ticket Sales Course For 2026 (4+ hours)
                    https://www.youtube.com/watch?v=AX5twE1ckQg

Alt:                The real difference between setter and closer
                    https://www.youtube.com/watch?v=-SeTLvEokJU

Suggested keyword:  THINK

CTA options:
 1. Most setters lose deals here. Comment THINK and I'll DM you the full ...
 2. Comment THINK and I'll send you the free course where I walk through ...
 3. I actually made a free 4-hour course on YouTube that covers this exact ...
 4. Save this video. Then comment THINK and I'll DM you the full objection ...
 5. Don't be most setters. Comment THINK and I'll DM you the full course ...
```

Step 3 — Pick one and lock it in

Tell Claude which option you want. Claude runs:

```
python3 ~/.claude/skills/cta-writer/scripts/save_keyword.py dyck \
  --keyword THINK \
  --video-id AX5twE1ckQg \
  --cta "I actually made a free 4-hour course on YouTube..." \
  --platform instagram
```

Two things happen:

1. The keyword is locked in `data/keywords.json`. Future CTAs for Dyck will never propose `THINK` again — no ManyChat conflicts.
2. You get a paste-ready ManyChat setup block:

```
═══════════════════════════════════════════════════════
 MANYCHAT SETUP — Instagram Comments → DM
═══════════════════════════════════════════════════════

Client:         Dyck (dyck)
Platform:       instagram
Trigger:        Comments on Post

📌 Post:         [paste post URL after publishing]
🔑 Keyword:      THINK   (match: contains, case-insensitive)

💬 Public comment reply:
   "In your DMs 🎯"

── DM STAGE 1 — Opening DM ──────────────────────────
Message:       Sliding it over — just tap below
Button label:  Gimme the link

── DM STAGE 2 — Follow-gate DM ──────────────────────
Message:       One more thing — this one's for the real ones. Follow me
               and I'll slide it into your DMs.
Button label:  Hit follow

── DM STAGE 3 — Link DM ─────────────────────────────
Message:       There you go — go eat
Link card:     Open it
URL:           https://www.youtube.com/watch?v=AX5twE1ckQg
Video title:   FREE High Ticket Sales Course For 2026 (4+ hours)

📝 CTA in the Reel:
   I actually made a free 4-hour course on YouTube that covers this exact
   objection and every other one you'll get on a call. Comment THINK and
   I'll DM it to you.

ManyChat path:  Instagram → Automation → New Automation → Comments → Instagram Post
═══════════════════════════════════════════════════════
```

Every time you run `save_keyword.py`, the comment reply and the three DM messages are randomly picked from a pool so your posts don't all feel like the same template. See the "Customizing" section further down if you want to edit the pool.

Step 4 — Paste into ManyChat

Open ManyChat → Instagram → Automation → **New Automation** → Comments → **Instagram Post**.

Fill in:

- **Post:** Paste the Instagram post URL once the Reel is published. (If you haven't published yet, save a draft or wait until it's up.)
- **Keyword:** `THINK` (match type: contains, case-insensitive)
- **Public comment reply:** copy-paste from the block
- **DM stage 1 (opening):** paste the message + button label
- **DM stage 2 (follow-gate):** paste the message + button label
- **DM stage 3 (link delivery):** paste the message, set the link card label, and paste the URL

Click **Go Live**.

Step 5 — Publish the Reel and you're done

Viewers who comment THINK trigger the whole sequence automatically.

Managing clients over time

See every registered client

```
python3 ~/.claude/skills/cta-writer/scripts/list_clients.py
```

Shows every client slug, their channel URL, when you last refreshed videos, and how many are cached.

See every keyword you've ever used for a client

```
python3 ~/.claude/skills/cta-writer/scripts/list_keywords.py dyck
```

Prints a table of keyword + date + which video it points to. Useful when you want to audit the whole history or remember what's been shipped.

Refresh every client at once

Useful to run weekly (or before a filming day):

```
python3 ~/.claude/skills/cta-writer/scripts/update_videos.py --all
```

Platforms supported

Platform	ManyChat automation	What you get
Instagram	Full	Complete 3-stage DM setup block
Facebook	Full	Complete 3-stage DM setup block
TikTok	Not supported	A manual-reply block (ManyChat doesn't do TikTok — you'd need a 3rd-party comment tool to automate)

Pass `--platform facebook` or `--platform tiktok` to `save_keyword.py` to override the default (instagram).

Customizing the ManyChat variation pool

The pool of random text for each slot lives at:

```
~/.claude/skills/cta-writer/references/manychat-variations.json
```

Seven slots:

- `comment_reply` — public auto-reply under the comment
- `opening_dm` — first DM after they comment
- `opening_button` — button label on that DM
- `follow_gate_dm` — message asking them to follow
- `follow_gate_button` — button label confirming they followed
- `link_dm` — message delivering the link
- `link_card_label` — the label on the link card

Edit freely. Add or remove variations. `save_keyword.py` picks one at random from each slot per run.

If you ever want to **lock** a specific line for a single run (e.g. match the voice of a particular client), use the override flags when running `save_keyword.py`:

```
python3 ~/.claude/skills/cta-writer/scripts/save_keyword.py dyck \
  --keyword THINK --video-id AX5twE1ckQg --cta "..." \
  --platform instagram \
  --comment-reply "Sent 📩 check ur DMs" \
  --opening-dm "Yo, tap the button and I'll send it" \
  --opening-button "Send it" \
  --gate-dm "Quick — follow me first and I'll send it right after" \
  --gate-button "Followed" \
  --link-dm "Here you go my g" \
  --link-label "Watch now"
```

Troubleshooting

`update_videos.py` says "no captions" for every video

YouTube occasionally requires a PO token for auto-captions. The skill uses `youtube-transcript-api`, which bypasses that limitation for most channels. If it genuinely fails for a video, that video just gets cached with `has_transcript: false` — Claude will skip it when matching and rely on title/description instead.

`keyword 'X' already used for <client>`

That keyword is locked for this client. Pick a new one — Claude will suggest alternatives automatically since it reads `keywords.json` before proposing keywords.

`client '<slug>' not registered`

Run `add_client.py` first.

Claude picks a long-form video that doesn't actually fit

Two causes:

1. The channel genuinely doesn't have a matching video yet — the short-form is ahead of the long-form. Consider using a generic "follow for more" CTA for this Reel, or film a matching long-form before posting.
2. The transcript for the best match is missing. Check `data/videos/<slug>.json` — any entry with `has_transcript: false` is matched on title only. Re-run `update_videos.py <slug>` to try again.

File layout reference

```
~/.claude/skills/cta-writer/
├── SKILL.md                        — rules Claude follows when invoked
├── README.md                       — quickstart
├── GUIDE.md                        — this document
├── references/
│   ├── cta-templates.csv           — proven CTA patterns Claude pulls from
│   └── manychat-variations.json    — rotated text for ManyChat setups
├── scripts/
│   ├── add_client.py               — register a client
│   ├── update_videos.py            — scrape/refresh videos + transcripts
│   ├── list_clients.py             — list registered clients
│   ├── list_keywords.py            — list used keywords per client
│   └── save_keyword.py             — lock keyword + print ManyChat block
└── data/                           — your private state (gitignored)
    ├── clients.json                — client registry
    ├── keywords.json               — used keywords per client
    ├── videos/<slug>.json          — cached video list per client
    └── transcripts/<slug>/<id>.txt — per-video transcript
```

Everything in `data/` is yours — it never goes to GitHub, it doesn't leave your machine.

Updating the skill when there's a new version

```
cd ~/.claude/skills/cta-writer
git pull
```

Your `data/` folder is untouched. Just the code/docs refresh.

One-line cheat sheet

```
1. Add client (once):  add_client.py "Name" <channel-url>  → update_videos.py <slug>
2. New CTA:            /cta-writer → paste script → pick option → save_keyword.py runs
3. ManyChat:           paste the block into IG → Automation → New → Comments
4. Weekly:             update_videos.py --all
```